



Department of IT and Computer Science
Pak-Austria Fachhochschule: Institute of Applied Sciences and
Technology, Haripur, Pakistan

SE-Red: DBS Grocery Store Management System Project

Submitted By:

Name	Registration Number
Haseeb Tahir	B24F0310SE138
Hamza Tanveer	B24F0302SE022
Abdul Mohiz Khan	B24F0141SE148

Submitted To:
Mr. Jawad Sadfar

Date of Submission:
May 3, 2026

Contents:

1. Introduction	3
2. Stack and Tools Used	3
3. Working of the Project	4
3.1 Login Page	4
3.2 Admin Side — Control Center	4
3.3 Sales Side — Sales Desk	8
4. Database Part	11
4.1 Database Tables	11
4.2 Stored Procedures	12
4.3 Triggers	13
4.4 Views	14
4.5 Joins	15
5. Challenges We Faced	16
5.1 Unclear Frontend Requirements	16
5.2 Connecting the Database to the Frontend	16
5.3 Implementing Triggers and Stored Procedures	16
5.4 Handling kg vs. Unit Products	17
5.5 Return Requests Flow	17
6. Conclusion	17

1. Introduction

This lab report documents our Grocery Store Management System, a full-stack web application we built as part of our database course. The basic idea behind this project was to create a system that a small grocery store could actually use to manage its daily operations, from handling sales to tracking inventory and suppliers.

The system has two main sides to it. One is the Admin panel, which is meant for the store owner or manager. The other is the Sales Desk, which is what the cashier or salesperson uses when dealing with customers at the counter.

The project covers a wide range of real-world features. The admin can manage the product catalog, create and manage staff accounts, place orders from suppliers when stock is low, review all sales receipts, handle return requests from customers, and see customer feedback and ratings. On the other hand, the salesperson can browse the product catalog, add items to a cart, handle customer checkouts with support for kg/gram-based and unit-based items, generate printed receipts, and submit return requests on behalf of customers.

The reason we chose this project was because it is database-centric, meaning almost every action in the system involves reading from or writing to the database. It gave us a chance to work with real concepts like stored procedures, triggers, views, and joins, not just simple SELECT queries.

2. Stack and Tools Used

Since this was our first time building a full-stack application, we kept the stack simple and focused on technologies that were practical to learn together.

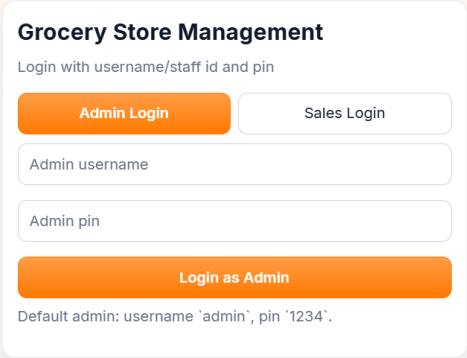
Layer	Technology	Purpose
Frontend	React.js	User interface, routing, component-based design
Backend	Express.js (Node.js)	REST API, business logic, DB communication
Database	MySQL	Data storage, procedures, triggers, views, Queries
IDE	Visual Studio Code	Code editing, debugging,

3. Working of the Project

The project is divided into two separate dashboards: Admin (Control Center) and Sales (Sales Desk). Both share the same backend and database.

3.1 Login Page

When you open the application, you see a login page with two tabs: Admin Login and Sales Login. The admin logs in using a username and PIN. The salesperson logs in using their Staff ID and PIN. The default admin credentials are username 'admin' and pin '1234'. Salesperson accounts are created by the admin from the Staff Members page.



The screenshot shows a login interface for 'Grocery Store Management'. It features a title, a subtitle 'Login with username/staff id and pin', and two tabs: 'Admin Login' (active) and 'Sales Login'. Below the tabs are input fields for 'Admin username' and 'Admin pin', followed by a 'Login as Admin' button. A footer note provides default admin credentials: 'Default admin: username 'admin', pin '1234'.'

3.2 Admin Side — Control Center

After logging in as admin, the user is taken to the Admin Dashboard (Control Center). The sidebar has links to all the admin sections.

Dashboard

The dashboard shows a daily summary of the store's performance. This includes: number of sales today, total revenue, total profit, total loss, approved returns today, and the number of low-stock items. Below the stats, there is a Low Stock Alerts section that shows which products are running low, and a Stock Notifications section that shows alert history.

Control Center

Administrator workspace

Dashboard

Catalog Management

Supplier Orders

Sales Receipts

Returns

Customer Feedback

Staff Members

Logout

Admin Dashboard

Daily business stats, profitability, and stock health based on live sales.

Sales Today

3

Revenue Today (PKR)

22,541

Profit Today (PKR)

4,091

Loss Today (PKR)

0

Approved Returns Today

0

Low Stock Items

2

Low Stock Alerts

Honey (Household Essentials)	8.000 left
Chicken (Meat)	14.000 left

Stock Notifications

Clear all checked notifications

Honey (Household Essentials) Stock: 8.000	Pending	Remove
---	----------------------	---------------------

This is where the admin can add new products, edit existing ones, or remove them. Products are organized by category (Dairy, Fruits, Vegetables, Meat, Dry Goods, Household Essentials). For each product the admin can set the product name, category, selling price, cost price, current stock, and whether the product is sold by unit or by weight (kg).

Control Center
Administrator workspace

Dashboard
Catalog Management
Supplier Orders
Sales Receipts
Returns
Customer Feedback
Staff Members
Logout

Catalog Management

Add, view, and remove catalog items category-wise.

Add New Item
Add Item

Category	Item Name	Sell Price	Cost Price	Stock	Action
Dairy	Milk 1L	PKR 280/unit	PKR 220/unit	30 unit	<button>Edit</button> <button>Remove</button>
	Yogurt	PKR 190/unit	PKR 140/unit	24 unit	<button>Edit</button> <button>Remove</button>
Fruits	Apple	PKR 260/kg	PKR 180/kg	18 kg	<button>Edit</button> <button>Remove</button>
	Banana	PKR 190/kg	PKR 110/kg	40 kg	<button>Edit</button> <button>Remove</button>
Vegetables	Potato	PKR 140/kg	PKR 95/kg	40 kg	<button>Edit</button> <button>Remove</button>
	Tomato 1kg	PKR 140/kg	PKR 95/kg	30 kg	<button>Edit</button> <button>Remove</button>
	Cheese 200g	PKR 380/unit	PKR 300/unit	60 unit	<button>Edit</button> <button>Remove</button>

Supplier Orders

If a product is running low, the admin can go to the Supplier Orders page and place a restock order. The admin selects a supplier from the dropdown, then selects a product linked to that supplier, enters the quantity, and clicks Place Order. When the order is placed, the product's stock in the catalog is automatically updated and the low-stock notification is resolved.

Control Center

Administrator workspace

Dashboard

Catalog Management

Supplier Orders

Sales Receipts

Returns

Customer Feedback

Staff Members

Logout

Supplier Orders Dashboard

Choose supplier, order products, and update catalog stock automatically.

Dairy Supplier

Choose supplier

Dairy Supplier

Dry Goods Supplier

Fresh Farm Supplier

Household Essentials Supplier

Meat Supplier

Choose product

20

Place Order

Chicken x 20	PKR 13000
Chicken x 20	PKR 13000
PO-1777802250280 Meat Supplier Chicken x 20	PKR 13000
PO-1777802237897 Meat Supplier Beef 1kg x 20	PKR 19800
PO-1777748674109 Meat Supplier Beef 1kg x 20	PKR 19800
PO-1777748656900 Dairy Supplier Cheese 200g x 20	PKR 6000
PO-1777748642847 Dairy Supplier Cheese 200g x 20	PKR 6000
PO-1777748534755 Meat Supplier Chicken x 20	PKR 13000
PO-1777720696136 Dry Goods Supplier Flour 10kg x 10	PKR 12500
PO-1777720228486 Household Essentials Supplier Shampoo Large x 20	PKR 4400

Sales Receipts

Every sale made from the Sales POS appears here. The admin can browse through all receipts and search by receipt number. Each receipt shows the customer name, phone, email, salesperson name, date and time, items ordered, discount, and total.

Control Center

Administrator workspace

Dashboard

Catalog Management

Supplier Orders

Sales Receipts

Returns

Customer Feedback

Staff Members

Logout

Sales Orders / Receipts

Every sale from POS appears here for admin review.

Search receipt by receipt number (e.g. REC-...)

REC-1777827846270

Card

Customer: Haseeb

Phone: 03161598773

Email: malikpdk123@gmail.com

Sales Staff: Unknown Staff (-)

Date: 5/3/2026, 12:04:06 PM

Ordered Items

Chicken x 1 kg

Subtotal: PKR 780

Discount: 5% (PKR 39)

Total: PKR 741

REC-1777804261141

Cash

Customer: Mohiz

Phone: -

Email: -

Sales Staff: Unknown Staff (-)

Date: 5/3/2026, 5:31:01 AM

Ordered Items

Honey x 6 unit

Subtotal: PKR 7,200

Discount: 0% (PKR 0)

Total: PKR 7,200

REC-1777802330072

Cash

Customer: Haseeb

Phone: -

Email: -

Sales Staff: Unknown Staff (-)

Date: 5/3/2026, 4:58:50 AM

Ordered Items

Chicken x 20 kg

Subtotal: PKR 14,600

Discount: 0% (PKR 0)

Total: PKR 14,600

REC-1777748745799

Cash

Customer: Haseeb

Phone: 12345678901

Email: malikpdk123@gmail.com

Sales Staff: Unknown Staff (-)

Date: 5/2/2026, 2:05:45 PM

Ordered Items

Subtotal: PKR 780

Discount: 5% (PKR 39)

Total: PKR 741

REC-1777745119252

Cash

Customer: Haseeb

Phone: 03161598773

Email: malikpdk123@gmail.com

Sales Staff: Unknown Staff (-)

Date: 5/2/2026, 1:05:19 PM

Ordered Items

Subtotal: PKR 7,200

Discount: 0% (PKR 0)

Total: PKR 7,200

REC-1777718796521

Card

Customer: Haseeb

Phone: 03161598773

Email: malikpdk123@gmail.com

Sales Staff: Haseeb (5)

Date: 5/2/2026, 5:46:36 AM

Ordered Items

Subtotal: PKR 14,600

Discount: 0% (PKR 0)

Total: PKR 14,600

Staff Members

The admin can create new salesperson accounts here by entering a name, staff ID, and PIN. They can also remove existing staff. For each staff member, it shows how many times they have logged in and when they last logged in.

Registered Sales Staff			
Sales Staff 1 ID: S001	Logins: 0 Never logged in		Remove
Haseeb ID: 5	Logins: 1 Last login: 5/2/2026, 7:50:01 PM		Remove
Haseeb ID: 1	Logins: 10 Last login: 5/3/2026, 5:03:27 PM		Remove
Sales Staff 2 ID: S002	Logins: 0 Never logged in		Remove

3.3 Sales Side — Sales Desk

The salesperson has three pages: POS / Checkout, Orders (their receipts), and Returns.

POS / Checkout

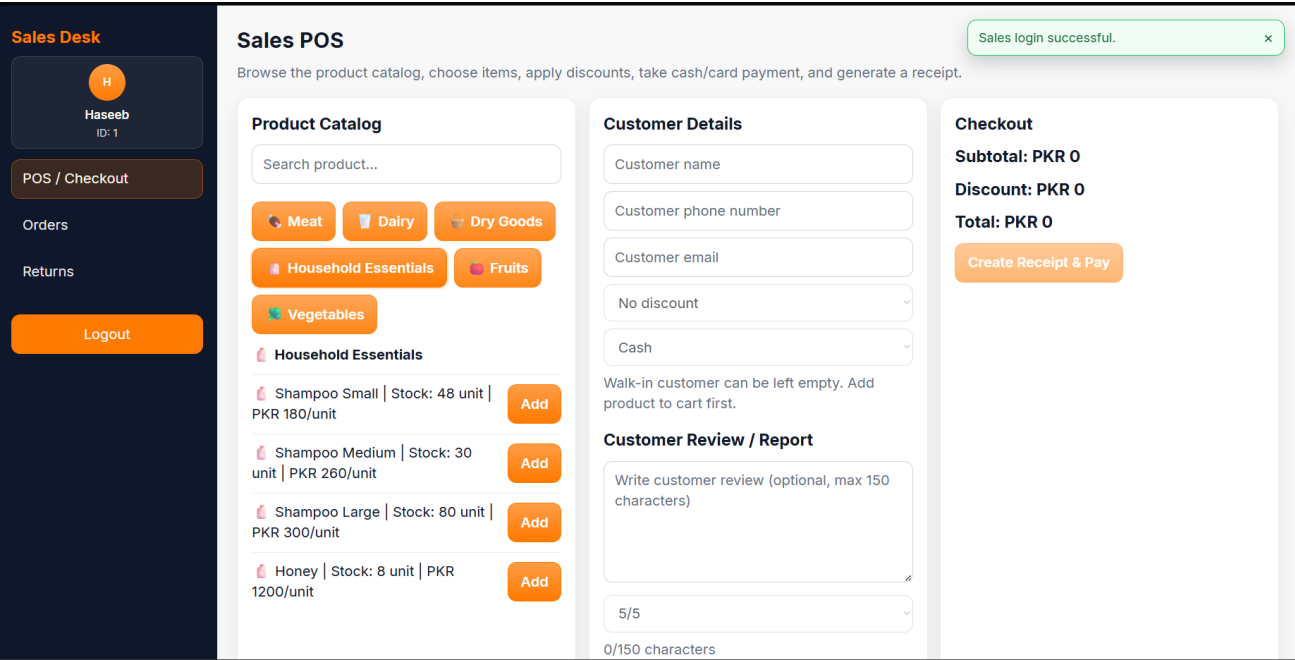
This is the main page for the salesperson. It has three columns: Product Catalog on the left, Customer Details in the middle, and Checkout on the right.

In the Product Catalog section, products are shown by category with buttons to filter. Each product shows its current stock and selling price. The salesperson clicks 'Add' to add a product to the cart. For weight-based items like Chicken or Beef, they can enter the quantity in kg or grams. For unit-based items, they enter a whole number. Products with sizes (like Shampoo Small, Shampoo Medium, Shampoo Large) are separate catalog entries.

In the Customer Details section, the salesperson can enter the customer's name, phone, and email (optional for walk-in customers). They can also select a discount percentage and choose the payment method (Cash or Card). There is also a Customer Review section where the customer can rate their experience (1 to 5 stars) and leave a comment.

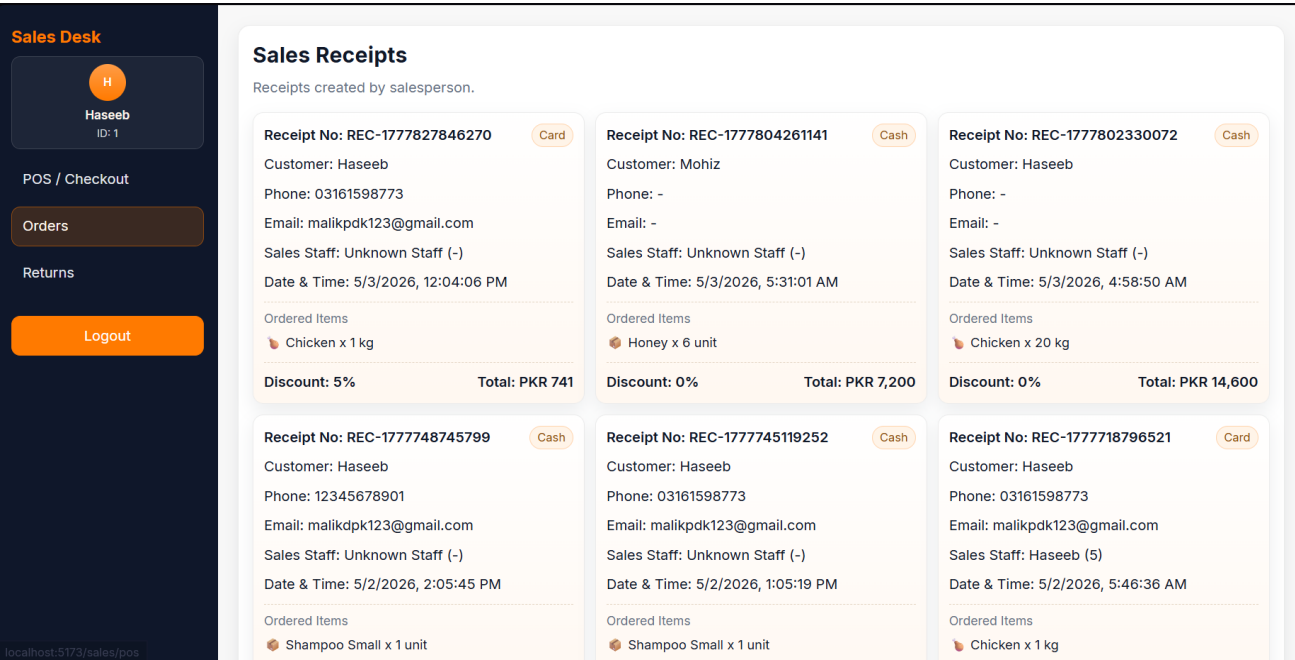
The Checkout section on the right shows the subtotal, discount amount, and final total. When ready, the salesperson clicks 'Create Receipt & Pay' to complete the sale. The receipt is generated and saved to the database.

Screenshot



Orders

This page shows all the receipts generated by the logged-in salesperson. The layout is similar to the admin's Sales Receipts page, but filtered to show only their own sales.



Returns

The salesperson enters a receipt number to look up a sale. Once the receipt is found, it shows the customer details and the items in that sale. The salesperson can then select which item to return, enter the quantity to return, and write the customer's reason. After submitting, the request goes to the admin for approval.

Sales Desk

H

Haseeb

ID: 1

POS / Checkout

Orders

Returns

Logout

Returns

Submit return requests using a receipt ID. Admin approval is required before refund.

Find Receipt

REC-1777827846270

Receipt: REC-1777827846270

Customer: Haseeb

Date: 5/3/2026, 12:04:06 PM

Total: PKR 741

Items

- Chicken x 1 kg

Create Return Request

Select item

1

IT WAS BAD

Submit Return Request

4. Database Part

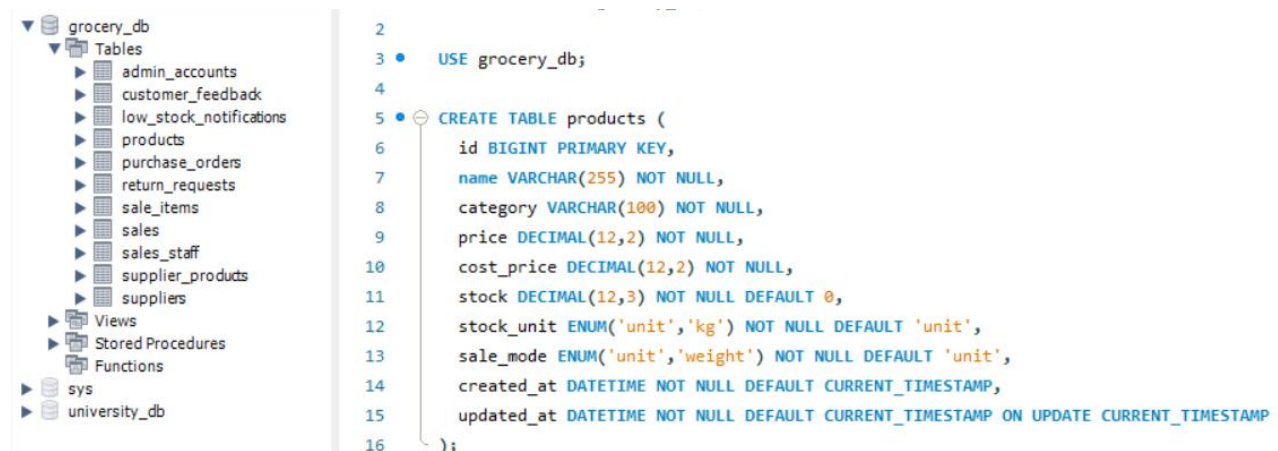
The database is the backbone of this project. Almost every button click in the UI results in a database query. We used MySQL and created 11 tables, along with stored procedures, triggers, views, and joins to keep things clean and efficient.

4.1 Database Tables

Below is a summary of all the tables in our database and what each one stores:

Table Name	Description
products	Stores all catalog items with name, category, selling price, cost price, stock, and sale mode (unit or weight).
sales_staff	Stores salesperson accounts with staff ID, PIN, name, and login history.
admin_accounts	Stores admin login credentials (username and PIN).
sales	Every completed sale with customer info, staff info, payment method, discount, totals, profit, and customer review.
sale_items	Each individual item in a sale (linked to sales and products). Stores quantity, unit, price at time of sale.
purchase_orders	Records every supplier restock order placed by the admin.
customer_feedback	Stores customer reviews and ratings submitted at checkout.
return_requests	All return requests with product, quantity, reason, status (pending/approved/rejected), and reviewer info.
low_stock_notifications	Auto-generated alerts when a product drops below 15 units/kg.
suppliers	List of suppliers with name and contact number.
supplier_products	Links suppliers to the products they supply, with cost price and default restock quantity.

Screenshot attached:



```
2
3 • USE grocery_db;
4
5 • CREATE TABLE products (
6     id BIGINT PRIMARY KEY,
7     name VARCHAR(255) NOT NULL,
8     category VARCHAR(100) NOT NULL,
9     price DECIMAL(12,2) NOT NULL,
10    cost_price DECIMAL(12,2) NOT NULL,
11    stock DECIMAL(12,3) NOT NULL DEFAULT 0,
12    stock_unit ENUM('unit','kg') NOT NULL DEFAULT 'unit',
13    sale_mode ENUM('unit','weight') NOT NULL DEFAULT 'unit',
14    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
15    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
16 );
```

4.2 Stored Procedures

We used stored procedures to wrap complex or multi-step database operations into reusable functions. Instead of writing long SQL from the backend every time, we just call the procedure with the needed parameters. Here is a summary:

Procedure Name	What It Does	Used In (Frontend)
sp_add_product	Inserts a new product into the catalog.	Admin - Catalog Management (Add Item)
sp_update_product	Updates an existing product's details.	Admin - Catalog Management (Edit Item)
sp_delete_product	Removes a product from the catalog.	Admin - Catalog Management (Remove Item)
sp_create_sales_staff	Creates a new salesperson account.	Admin - Staff Members (Create Staff ID)
sp_remove_sales_staff	Deletes a salesperson account.	Admin - Staff Members (Remove)
sp_restock_from_supplier	Places a supplier order and updates product stock. Also adds a purchase order record.	Admin - Supplier Orders (Place Order)
sp_create_sale	Records a completed sale into the sales table.	Sales POS (Create Receipt & Pay)
sp_add_customer_feedback	Saves a customer review and rating.	Sales POS (Customer Review section)
sp_create_return_request	Creates a new return request in the database.	Sales Desk - Returns (Submit Return Request)
sp_approve_return_request	Marks a return request as approved.	Admin - Returns (Approve button)
sp_reject_return_request	Marks a return request as rejected.	Admin - Returns (Reject button)

Database Screenshot:

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' panel displays a tree view of the 'grocery_db' database, including tables, views, and stored procedures. The 'Stored Procedures' folder is expanded, showing a list of procedures including 'sp_add_customer_feedback', 'sp_add_product', 'sp_approve_return_request', 'sp_create_return_request', 'sp_create_sale', 'sp_create_sales_staff', 'sp_delete_product', 'sp_reject_return_request', 'sp_remove_sales_staff', 'sp_restock_from_supplier', and 'sp_update_product'. The main editor on the right displays the SQL code for the 'sp_update_product' procedure. The code starts with a comment line, followed by 'DELIMITER ;', then 'DELIMITER //' and the 'CREATE PROCEDURE' statement. The procedure parameters are: 'IN in_id BIGINT', 'IN in_name VARCHAR(255)', 'IN in_category VARCHAR(100)', 'IN in_price DECIMAL(12,2)', 'IN in_cost_price DECIMAL(12,2)', 'IN in_stock DECIMAL(14,3)', and 'IN in_sale_mode ENUM('unit','weight')'. The procedure body begins with 'BEGIN', followed by 'UPDATE products' and 'SET'.

```
220 //
221 DELIMITER ;
222
223 DELIMITER //
224 CREATE PROCEDURE sp_update_product(
225     IN in_id BIGINT,
226     IN in_name VARCHAR(255),
227     IN in_category VARCHAR(100),
228     IN in_price DECIMAL(12,2),
229     IN in_cost_price DECIMAL(12,2),
230     IN in_stock DECIMAL(14,3),
231     IN in_sale_mode ENUM('unit','weight')
232 )
233 BEGIN
234     UPDATE products
235     SET
```

4.3 Triggers

Triggers are automatic actions that run in the database whenever something changes. We created 3 triggers:

Trigger Name	When It Fires	What It Does	Used In (Frontend)
trg_sale_item_after_insert	AFTER INSERT on sale_items	Automatically reduces product stock by the quantity sold.	Fires when a sale is completed at POS checkout.
trg_return_request_after_update	AFTER UPDATE on return_requests	When a return is approved, it adds the returned quantity back to the product's stock.	Fires when admin approves a return on the Returns page.
trg_product_low_stock_after_update	AFTER UPDATE on products	If a product's stock drops below 15, it automatically creates a low stock notification.	Fires after a sale is completed, detected via stock update.

Database Screenshot:

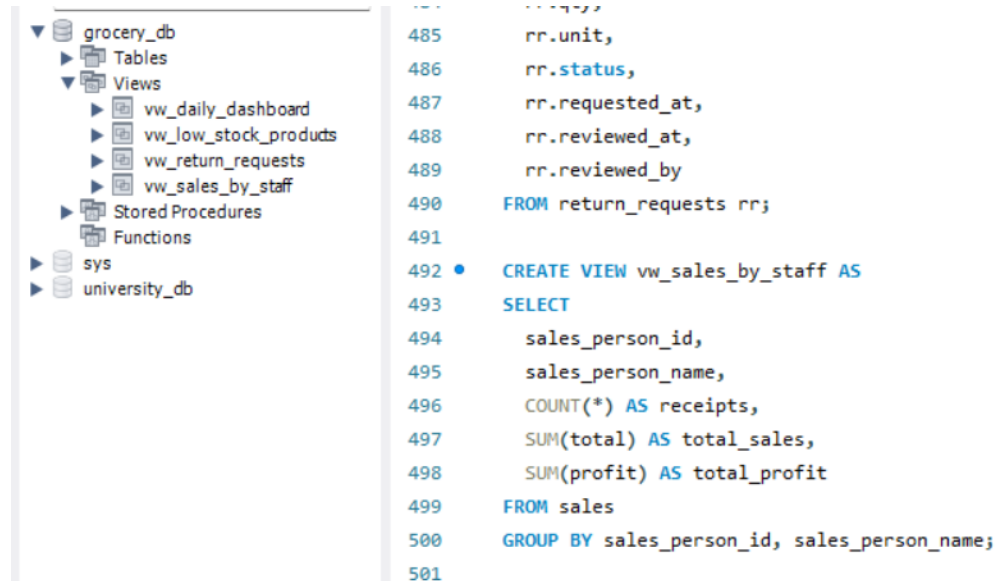
```
515 • CREATE TRIGGER trg_return_request_after_update
516 AFTER UPDATE ON return_requests
517 FOR EACH ROW
518 BEGIN
519 IF OLD.status = 'pending' AND NEW.status = 'approved' THEN
520 UPDATE products
521 SET stock = stock + NEW.qty_base
522 WHERE id = NEW.product_id;
523 END IF;
524 END;
525 //
526 DELIMITER ;
527
```

4.4 Views

Views are pre-written queries saved in the database that we can query like a regular table. They save us from writing complex SQL repeatedly. We created 4 views:

View Name	What It Returns	Used In (Frontend)
vw_daily_dashboard	Today's sales count, total revenue, total profit (minus approved refunds), and total loss. This is calculated using a subquery inside the view.	Admin Dashboard — the 4 stat cards at the top.
vw_low_stock_products	All products with stock below 15.	Admin Dashboard — Low Stock Alerts section and the Low Stock Items count.
vw_return_requests	All return requests with key info like receipt ID, product, status, and reviewer.	Admin Returns page to list all requests.
vw_sales_by_staff	Per-staff breakdown of number of receipts, total sales, and total profit using GROUP BY.	Can be used for staff performance reporting.

Database Screenshot:



4.5 Joins

We used JOIN queries in several places to combine data from multiple tables. Some examples:

- When loading the Supplier Orders page, we use a LEFT JOIN between supplier_products and suppliers to show each product alongside its supplier name.
- When displaying Sales Receipts, the backend joins the sales table with sale_items to show the full receipt with all ordered products.
- When loading return requests, we join return_requests with products to get up-to-date product details.
- The purchase_orders table has a foreign key to suppliers and we join them when showing supplier order history.

The use of FOREIGN KEYs across tables (sale_items referencing sales and products, return_requests referencing products and sales, supplier_products referencing suppliers) ensures data integrity. If a product is deleted for example, it cannot be deleted if there are existing sale_items referencing it.

Database Screenshot:

```
559 • SELECT * FROM suppliers;
560 • SELECT * FROM supplier_products;
561
562 • SELECT sp.*, s.name AS supplier
563 FROM supplier_products sp
564 LEFT JOIN suppliers s ON sp.supplier_id = s.id;
565
566 • INSERT INTO supplier_products (
567     supplier_id,
568     product_name,
569     category,
```

5. Challenges We Faced

This was our first full-stack project and we ran into a lot of issues along the way. Below are the main ones:

5.1 Unclear Frontend Requirements

At the start, we did not have a clear picture of what the UI should look like. We kept changing the layout and going back and forth on what features to include. After a few redesigns, we settled on the layout you see in the screenshots.

Eventually we decided to just start building and fix things as we went. That approach actually worked better for us than trying to plan everything upfront.

5.2 Connecting the Database to the Frontend

This was by far the hardest part of the project and took more than 3 days to get working. The frontend (React) and the backend (Express) are two separate processes. Getting them to communicate properly was tricky because of CORS errors, wrong API URL configurations, and mismatches between what the frontend expected and what the backend was returning.

We also struggled with `async/await` in JavaScript. When we first tried fetching data from the backend, we would sometimes render the page before the data was ready, which caused the screen to show empty or broken states.

Once we figured out that the React frontend needs to send requests to the Express server (not directly to MySQL), and that Express is the one that talks to MySQL, things became clearer. We also had to configure a proxy in the React app or set the full URL so requests would reach Express correctly.

5.3 Implementing Triggers and Stored Procedures

Writing stored procedures and triggers in MySQL was something we had not done before in practice. Understanding the `DELIMITER` syntax, how to write `IF/ELSE` logic inside procedures, and how triggers fire automatically took some time to get right.

One specific problem we had was with the stock update trigger. At first the trigger would fire but not update the right product because we forgot to use `NEW.product_id` correctly. After reading the MySQL documentation more carefully, we were able to fix it.

5.4 Handling kg vs. Unit Products

Some products like Chicken, Beef, and Bananas are sold by weight (you enter 0.5 kg or 250 grams), while others like Shampoo are sold by the size. Making the cart handle both of these in the same UI, and then storing them correctly in the database with the right unit and quantity, was a challenge.

5.5 Return Requests Flow

The return request feature had multiple steps: the salesperson finds a receipt, selects the item to return, submits the request, and then the admin has to approve it. Getting the status changes and stock restoration to work correctly through the trigger was a bit tricky, especially making sure the trigger only fires when the status changes from 'pending' to 'approved' and not on other updates.

6. Conclusion

Overall, building this Grocery Store Management System was a great learning experience for us. It was our first time working with a full-stack setup where the frontend, backend, and database all talk to each other, and it was definitely challenging.

We learned a lot about how real web applications work. The database side was the most educational part. Using stored procedures made our backend code much cleaner because instead of writing raw complex SQL in the Express routes, we just called a procedure. Triggers were especially interesting because they do their work automatically without the backend having to explicitly tell the database to update stock or create a notification. Views made certain dashboard queries much simpler.

The biggest lesson we took from this project is that connecting the different layers of a full-stack app is harder than it looks. It is not just about writing code but also about understanding how the frontend, backend, and database communicate, what data format to use, and how to handle errors at each step.

If we had more time, we would add things like PDF receipt generation, monthly sales charts for the admin dashboard, and a proper authentication system with tokens instead of just PIN-based login.

All features shown in the screenshots are fully working and connected to the live MySQL database. The project runs locally and demonstrates a real-world use case of a database-centric full-stack application.

Thank you.
